



**Mondragon
Unibertsitatea**

**Escuela Politécnica
Superior**

Reinforcement Learning -RL-

Hyperparameter Optimization

Aitor Duo Zubiaurre

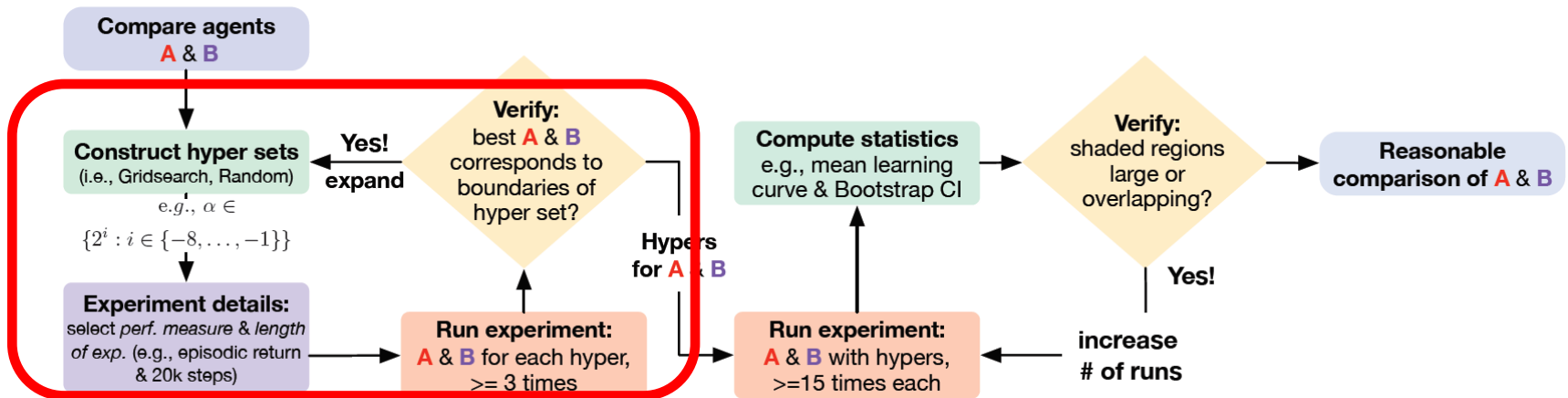
aduo@mondragon.edu

Olagorta Kalea, 26
48014 Bilbo (Bizkaia)

T. +34 670 254 555

www.mondragon.edu

Hyperparameter Optimization

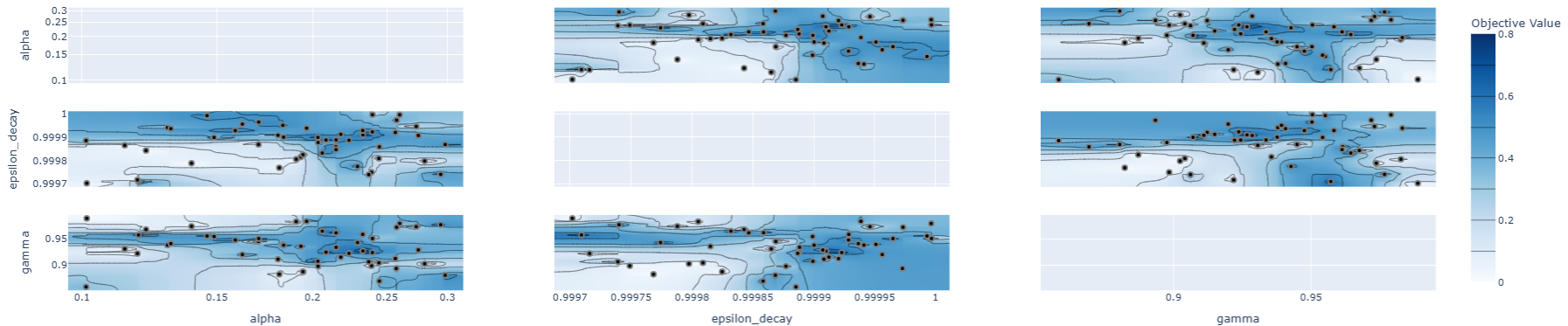
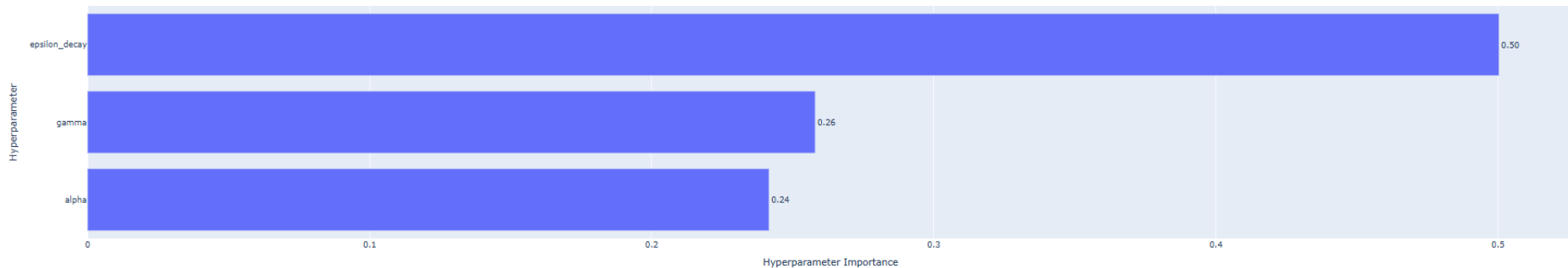


Patterson, A., Neumann, S., White, M., & White, A. (2023). Empirical Design in Reinforcement Learning. *Journal of Machine Learning Research*, 25, 1–63.

Hyperparameter Optimization

- What are hyperparameters?
 - Hyperparameters control the behaviour of algorithms when creating models.
 - Determine performance
 - Typically set manually or (randomized search, grid search)

Hyperparameter Importances



Hyperparameter Optimization

- In RL there are large number of hyperparameters that can have large effect on performance.
 - Classic RL
 - Discount factor γ
 - Learning rate α
 - Epsilon decay
 - Steps
 - Deep RL
 - Experience replay: buffer size & batch size
 - Learning rate
 - Number of layers
 - Neurons per layer
 - ...

Hyperparameter Optimization

- How to choose hyperparameters
 - **Objective:** average cumulative reward
 - Hyperparameter tuning
 - Trial and error by hand
 - Grid search
 - Random search
 - Dedicated algorithms

Hyperparameter Optimization

- **Optuna workflow**



<https://optuna.org/>

```
import optuna
```

```
def objective(trial: optuna.Trial):
```

```
    ... #your code here
```

→ Define objective function

```
study = optuna.create_study()
```

```
study.optimize(objective, n_trials=100)
```

→ Optimize

```
print("  Params: ")
```

```
for key, value in trial.params.items():
```

```
    print(f"    {key}: {value}")
```

→ Show best
hyperparameters

Hyperparameter Optimization

- **Objective function**

```
def objective(trial: optuna.Trial) -> float:
```

```
    hyperparams = sample_Qlearning_params(trial)
```

```
    print(hyperparams)
```

→ Sample hyperparameters

```
    env = gym.make('Taxi-v3', max_episode_steps=TIME_LIMIT)
```

```
    model = Q_learning(env=env, gamma=hyperparams['gamma'],
```

```
                        alpha=hyperparams['alpha'],
```

```
                        epsilon=EPSILON,
```

```
                        min_epsilon=MIN_EPSILON,
```

```
                        epsilon_decay=hyperparams['epsilon_decay'])
```

```
    model.train(num_episodes=10_000)
```

```
    mean_reward, _ = model.evaluate_policy(num_episodes=100)
```

```
    return mean_reward
```

→ Train model on environment

→ Evaluate

Hyperparameter Optimization

- Sampling hyperparameters

```
def sample_Qlearning_params(trial: optuna.Trial):
    # Suggest hyperparameters
    alpha = trial.suggest_float("alpha", 0.01, 0.3, log=True)
    gamma = trial.suggest_float("gamma", 0.8, 1.0)
    epsilon_decay = trial.suggest_float("epsilon_decay", 0.9, 1.0)
    return {
        "alpha": alpha,
        "gamma": gamma,
        "epsilon_decay": epsilon_decay,
    }
```

<code>should_prune()</code>	Suggest whether the trial should be pruned or not.
<code>suggest_categorical()</code>	Suggest a value for the categorical parameter.
<code>suggest_discrete_uniform(name, low, high, q)</code>	Suggest a value for the discrete parameter.
<code>suggest_float(name, low, high, *[, step, log])</code>	Suggest a value for the floating point parameter.
<code>suggest_int(name, low, high, *[, step, log])</code>	Suggest a value for the integer parameter.
<code>suggest_loguniform(name, low, high)</code>	Suggest a value for the continuous parameter.
<code>suggest_uniform(name, low, high)</code>	Suggest a value for the continuous parameter.

Hyperparameter Optimization

- **Optuna study**

- A study in Optuna is an object that manages the entire hyperparameter optimization process.

```
study = optuna.create_study()
```

```
study=optuna.create_study(direction="maximize",
```

This is the best way to
store results

```
storage="sqlite:///DQN.db",  
study_name="study_name",  
load_if_exists=True)
```

In this case
we want to
maximize the
return

If the study exists we can continue
from the last trial

Hyperparameter Optimization

- **Optimize**

```
study.optimize(objective,  
               n_trials=N_TRIALS,  
               n_jobs=N_JOBS)
```

The objective function we want to optimize

If omitted run trials until interrupt

Beware of this parameter

- In most cases, using `n_jobs > 1` with GPU computations doesn't make sense and can actually be counterproductive
- When using a CPU with more than 1 core, `n_jobs=n_cores-1`, to leave a core for system processes

Hyperparameter Optimization

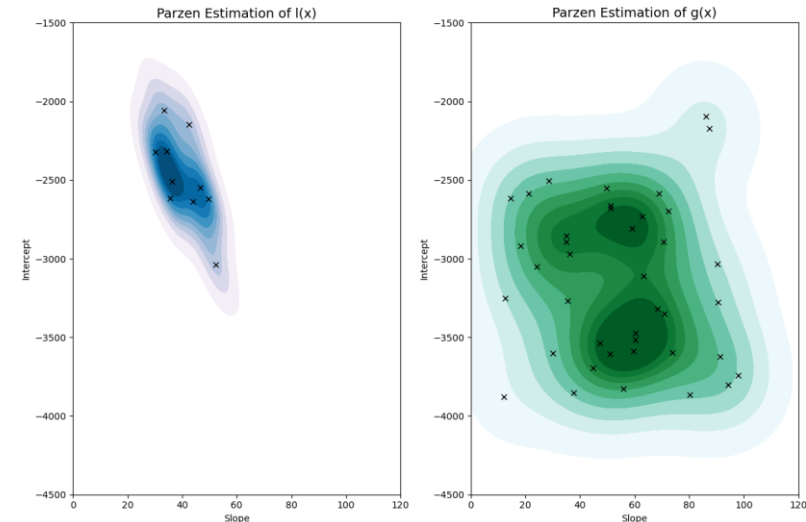
- Samplers

	RandomSampler	GridSampler	TPESampler	CmaEsSampler	NSGAIISampler
Float parameters	✓	✓	✓	✓	▲
Integer parameters	✓	✓	✓	✓	▲
Categorical parameters	✓	✓	✓	▲	✓
Pruning	✓	✓	✓	▲	✗ (▲ for single-objective)
Multivariate optimization	▲	▲	✓	✓	▲
Conditional search space	✓	▲	✓	▲	▲
Multi-objective optimization	✓	✓	✓	✗	✓ (▲ for single-objective)
Batch optimization	✓	✓	✓	✓	✓
Distributed optimization	✓	✓	✓	✓	✓
Constrained optimization	✗	✗	✓	✗	✓
Time complexity (per trial) (*)	$O(d)$	$O(dn)$	$O(dn \log n)$	$O(d^3)$	$O(mp^2)$ (***)
Recommended budgets (#trials) (**)	as many as one likes	number of combinations	100 - 1000	1000 - 10000	100 - 10000

Hyperparameter Optimization

- **TPE Sampler**

1. The first step of TPE is to randomly sample sets of hyperparameters
2. Split our hyperparameter search space in two using a quantile threshold γ , where γ is between 0 and 1.
3. Determine the next “Best” hyperparameters to test



```
sampler = TPESampler(n_startup_trials=N_STARTUP_TRIALS)
```

The **Tree-structured Parzen Estimator (TPE) Sampler** is commonly used in **reinforcement learning (RL)** for hyperparameter optimization because of its ability to efficiently explore and exploit the hyperparameter search space.

Hyperparameter Optimization

- **Median Pruner**

- Is a pruning strategy that stops unpromising trials early based on median performance.
- It works by comparing the current trial's performance against the median performance of previous trials at the same step.
- If the current trial performs worse than the median, it gets pruned (stopped early) to save time and resources.

```
pruner = MedianPruner(n_startup_trials=N_STARTUP_TRIALS,  
n_warmup_steps=N_EVALUATIONS // 3  
)
```

Hyperparameter Optimization

- Usage

```
sampler = TPESampler(n_startup_trials=N_STARTUP_TRIALS)
pruner = MedianPruner(n_startup_trials=N_STARTUP_TRIALS,
n_warmup_steps=N_EVALUATIONS // 3)
study = optuna.create_study(sampler=sampler,
                             pruner=pruner,
                             direction="maximize",
                             storage="sqlite:///DQN.db",
                             study_name="study_name",
                             load_if_exists=True)
```

Hyperparameter Optimization

- Visualization
- <https://optuna.github.io/optuna-dashboard/>

▼ hoy (6)

 DQN_cartpole.db	26/02/2025 19:18	Data Base File	112 KB
 DQN (18).db	26/02/2025 19:17	Data Base File	176 KB

Drag and Drop

Optuna Dashboard (Wasm ver.)

🔍 Search study

☰ Sort ascending ▼



Load an Optuna Storage

Drag your SQLite3/JournalStorage file here or click to browse.

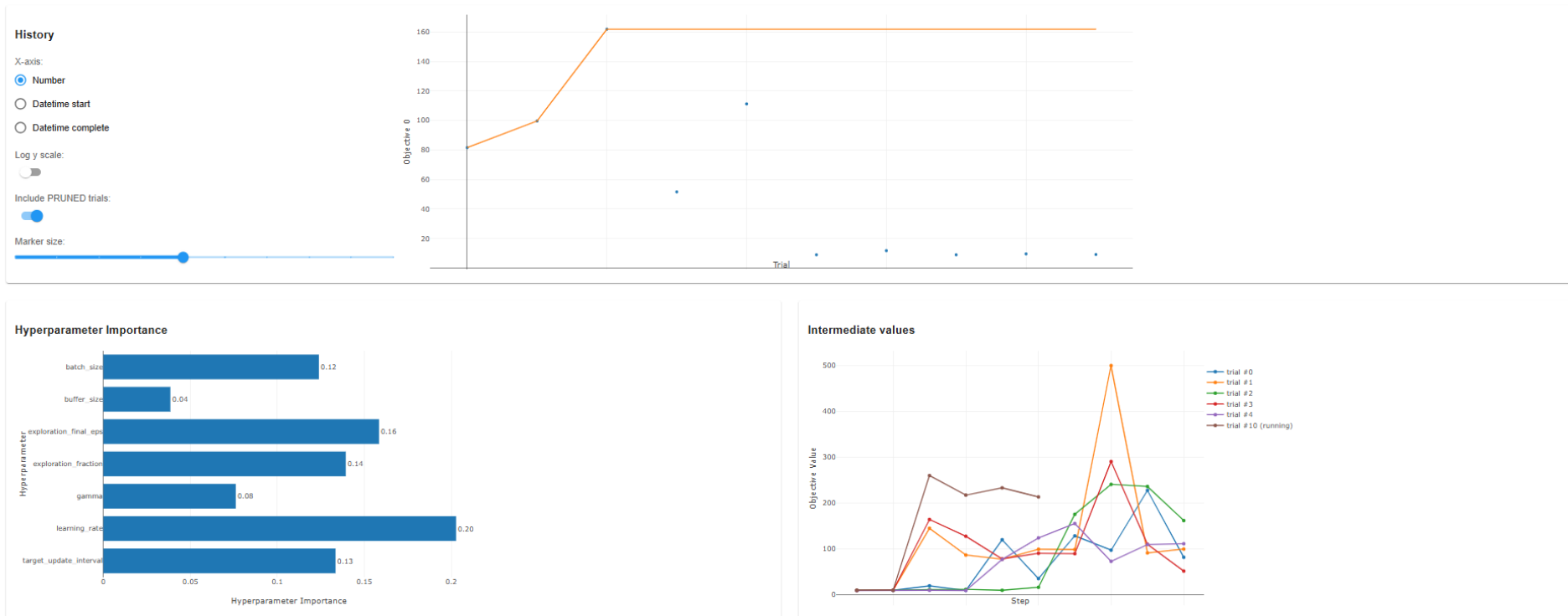
Hyperparameter Optimization

- Visualization

Optuna Dashboard



cartpole_v1



Hyperparameter Optimization

- **Summary**
 - In deep RL where the number of hyperparameters is large.
 - Optuna is a powerful tool for automating hyperparameter tuning, with features like samplers and pruners that enhance efficiency.
 - The use of visualization tools helps in understanding the optimization process and making informed decisions.



**Mondragon
Unibertsitatea**

Escuela Politécnica
Superior

Eskerrik asko
Muchas gracias
Thank you

© 2026 Mondragon Unibertsitatea.

All rights reserved. This document and its contents are protected by intellectual property rights. Its total or partial reproduction, distribution, public communication, transformation, adaptation or creation of derivative works is expressly prohibited without the written consent of the owner.

Aitor Duo Zubiaurre

aduo@mondragon.edu

Olagorta Kalea, 26
48014 Bilbo (Bizkaia)

T. +34 670 254 555

www.mondragon.edu